Artificial Intelligence
Introduction and examples

Stefaan Haspeslagh
Vives University of applied sciences – Association KU Leuven – Kortrijk, Belgium

Artificial Intelligence: introduction and examples

COMPARISON OF ALGORITHMS

Blind search

- We have:
  - State-space representation: how to formulate problem, goal, production rules, …
  - 2 algorithms to ‘search the tree’
    - Breath-first
    - Depth-first
- Question in this topic
  - What algorithm is best?

Blind search

- How to compare algorithms?
- We need measures!
- We focus on three criteria:
  - Completeness
  - Speed
  - Memory

De centrale vraag die we ons stellen is hoe we de algoritmen, breedte-eerst en diepte-eerst uit het vorige topic, kunnen vergelijken.
Algemeen gezien, en dit is niet alleen voor algoritmen zo, is er om een goede vergelijking op te stellen nood aan goede criteria, kenmerken of eigenschappen. Als je processoren vergelijkt, ga je bv. kijken naar de kenmerken kloksnelheid en aantal kernen.
Om algoritmen te vergelijken hebben we dus ook dergelijke kenmerken nodig.
In dit topic schuiven we drie kenmerken naar voor: completeness, speed en memory.
In de volgende slide lichten we deze kenmerken toe.

Evaluation criteria

- Completeness
  - Does the algorithm always find a path?
- Speed
  - What is the highest number of paths/nodes that may need to be created?
- Memory
  - What is the largest amount of nodes/paths that may need to be stored?

katholieke hogeschool vives
associatie KU Leuven

Completeness

Bij completeness, of volledigheid, proberen we in te schatten of het algoritme altijd een oplossing vindt. Dit is niet zomaar het geval voor elke mogelijke zoekboom. Indien het algoritme in staat is altijd een oplossing te vinden, dan zeggen we dat het algoritme compleet is.

Speed

Speed gaat over de rekentijd die nodig is om een oplossing te vinden. De algoritmen breedte-eerst en diepte-eerst zijn beiden gebaseerd op hetzelfde principe: ze genereren paden voor alle kinderen, en als ze een pad beschouwen, controleren ze of het pad het doel bereikt. Enkel de volgorde waarin de paden bezocht worden is verschillend.
Hoe meer paden er moeten gegenereerd worden, hoe meer tijd het algoritme nodig heeft om het doel te vinden.
De rekentijd van een algoritme proberen we dus in te schatten aan de hand van het maximaal aantal, het grootste aantal paden die door het algoritme moet berekend worden.

Memory

Analoog aan snelheid, proberen we in te schatten hoeveel geheugen de algoritmen nodig hebben, dus het grootste, of maximaal aantal paden dat in het geheugen moet opgeslagen worden.

Evaluation criteria

- How to measure the criteria?
- We use what we have → the search tree
- Express the criteria in terms of:
  - d = depth of the tree
  - b = (average) branching factor of the tree
  - m = depth of the shallowest solution

In de vorige slide hebben we een beschrijving gegeven wat we onder de verschillende kenmerken begrijpen.
We moeten echter in staat zijn, vooral voor snelheid en geheugen, om effectief een getal te plakken op de rekentijd en het geheugengebruik.

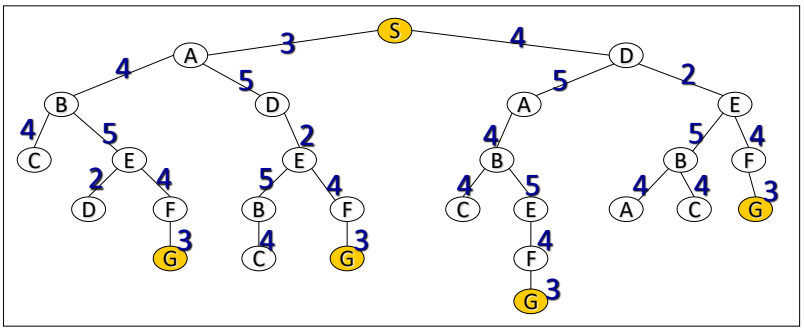
We gebruiken daarvoor wat we ter beschikking hebben: de structuur van de zoekboom.

Drie waarden zijn daarbij belangrijk:
- d, de diepte van de boom
- b, de (gemiddelde) branching factor van de boom
- m, de diepte van de minst diepe oplossing.

Op de volgende slide geven we een voorbeeld van deze drie getallen.

Example of d, b and m

- d = 6
- b = 2
- m = 4



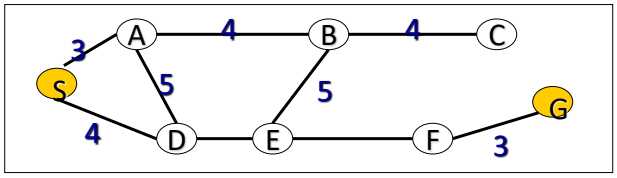
In deze figuur zijn er 7 niveaus, maar we starten bij 0 en komen zo bij d = 6 uit (niveaus 0 t.e.m. 6). De branching factor van de zoekboom is gemiddeld 1,53 maar we ronden af naar 2, en de diepte van de minst diepe oplossing (minst diepe node G) is 4 (want die node bevindt zich op het 5de niveau startend van niveau 0).

BLIND SEARCH

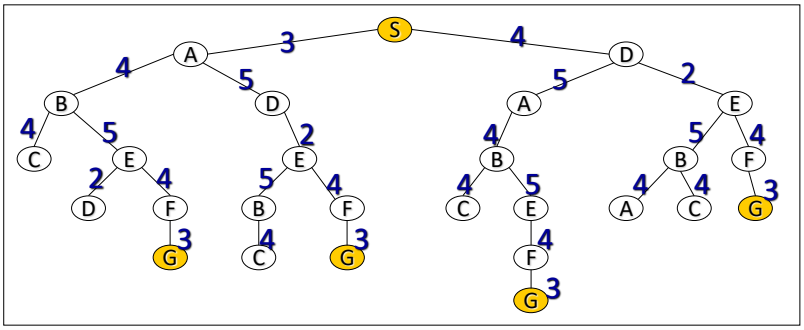
CRITERIA FOR DEPTH-FIRST

In de volgende slides bestuderen we de criteria voor het algoritme diepte-eerst.

Completeness for depth first



- Complete for FINITE nets
- Important: due the LOOP-checking!
  - If we do not remove the paths with loops, depth-first may be trapped in a loop
- Note: does not find the shortest path



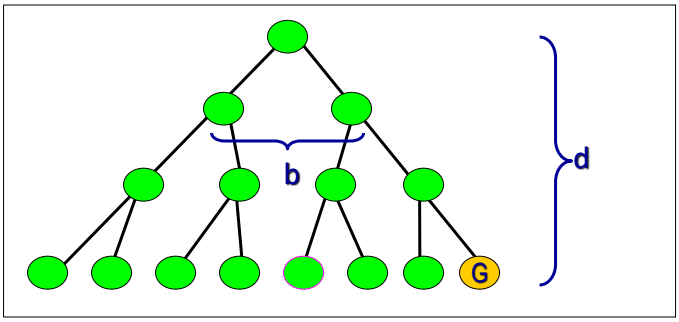
Het algoritme depth-first is compleet onder enkele voorwaarden.

- Ten eerste mag de spelboom geen oneindige takken bevatten. Het netwerk of de boom moet eindig zijn (Finite net).
- Ten tweede mogen er geen lussen voorkomen! Indien diepte-eerst eenmaal een lus genomen heeft, dat zal hij telkens opnieuw in die lus terecht komen. Je kan dit zelf zien door het algoritme van het vorige topic uit te voeren zonder loop-checking.

Let op: het algoritme vindt niet het kortste pad naar knoop G. Het pad dat door diepte-eerst gevonden wordt is (SABEFG) met lengte 5. Er is echter een korter pad, als je telkens de laatste tak neemt, verkrijg je het pad (SDEFG) met lengte 4.

Speed for depth first

- Worst case: the goal node may be on the right-most branch
- Time complexity: $1 + \ldots + b^{d-1} + b^d = \frac{b^{d+1}-1}{b-1}$
- Thus: $O(b^d)$



In deze slide pogen we de rekentijd van het diepte-eerst algoritme in te schatten.
We hebben eerder al gezegd dat de rekentijd geschat kan worden met behulp van het aantal nodes/paden dat gegenereerd wordt.

We gaan uitgaan van het slechts mogelijke geval. Namelijk dat het doel zich helemaal onderaan in de meest rechtse tak bevindt.
Dan moeten alle nodes in de boom bezocht worden door diepte-eerst voordat de oplossing gevonden kan worden.
Dan moeten we niveau per niveau het aantal nodes tellen.

Op het nulde niveau (we starten in computerwetenschappen meestal vanaf 0) tellen we 1 knoop, de startknoop. Vanuit deze knoop worden b nieuwe knopen gegenereerd. (b is de branching factor, die aangeeft hoeveel nieuwe takken er uit een knoop vertrekken, zie vorig topic).
In de figuur is de branching factor twee. Dus na “twee stappen” in het algoritme werden er 1 + 2 = 3 knopen gegenereerd.

Per knoop van het eerste niveau, worden opnieuw b nieuwe knopen/paden gegenereerd. Dus op het tweede niveau zijn er in de figuur 4 nieuwe knopen die gegeneerd worden. Algemeen wordt op niveau i, b^i knopen gegenereerd. Dus
- niveau 0: $b^0 = 1$ knoop
- Niveau 1: $b^1 = b$ knopen, in het voorbeeld dus 2

- Niveau 2: b^2 = b * b knopen, in het voorbeeld dus 4
- …

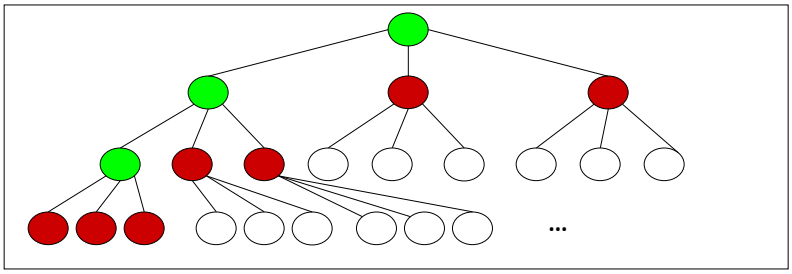
Tellen we al die knopen op voor alle niveaus, dan bekomen we de formule zoals op de slide.

Als we een complexiteit inschatten, zijn we nooit geïnteresseerd in de exacte tijd of in het exact aantal gegenereerde paden of nodes, maar in een orde-grootte. Daarom gebruik we de O notatie. Die toont de trend aan van het aantal gegenereerde nodes. Daarmee kunnen we vragen oplossen zoals:

- Als er een niveau extra bijkomt, hoeveel meer nodes zijn er dan die gegenereerd moeten worden
- Als de branching factor toeneemt, wat is de invloed op het aantal te genereren nodes/paden…

Memory for depth-first

- Largest number of nodes in QUEUE is reached in bottom left-most node
- Example: d = 3, b = 3



- QUEUE contains all red nodes, thus 7
- In general: ((b-1)*d) + 1
- Order: O(d*b)

Bij het criterium memory proberen we het maximaal aantal nodes/paden te tellen die in het geheugen moeten opgeslagen worden.

Bij diepte eerst wordt dit aantal bereikt als de nodes/paden in de meest linker tak gegenereerd worden. Op dat moment zitten alle rode nodes in de QUEUE (Herinner: telkens een groene node bezocht wordt, dan wordt die verwijderd uit de QUEUE en vervangen door zijn kinderen).

Indien de rode nodes van de diepste linker groene node allemaal bezocht geweest zijn, dan worden die drie verwijderd uit de queue en vervangen door de drie 'kinderen' van de rode node naast die groene node, en zo verder…)

In het algemeen kunnen we dus stellen:
- Op het diepste niveau zitten er maximaal b nodes in de QUEUE, in het voorbeeld dus 3 (b is de branching factor!)
- Op de andere niveau's zitten maximaal b – 1 nodes in de QUEUE, want van 1 node zitten enkel de kinderen in de QUEUE, niet de node zelf!

Algemeen zit er dus op elk niveau minstens (b-1) nodes in de QUEUE, en dit voor elke niveau, vandaar (b-1) * d. Op het laagste niveau hebben we 1 node te weinig geteld, dus in totaal bekomen we ((b-1) * d) + 1.

Hier gebruiken we weer de O-notatie aangezien we niet in het exacte aantal geïnteresseerd zijn, maar in de grootteorde.

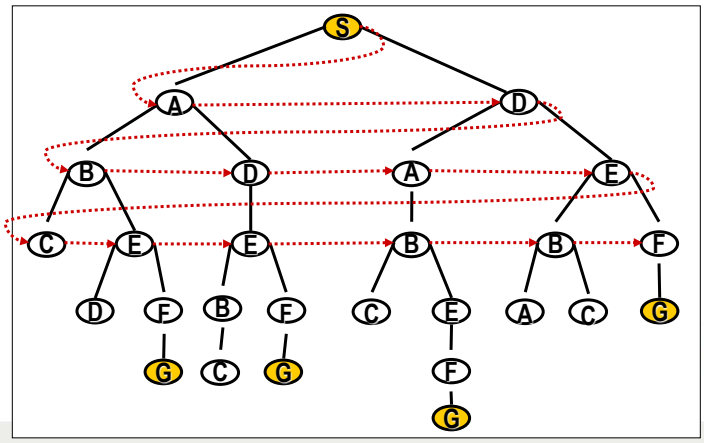
BLIND SEARCH

CRITERIA FOR BREADTH-FIRST

In de volgende slides herhalen we het meten van de criteria voor breedte-eerst.

Completeness for breadth first

- Complete, even for infinite nets
- Would remain complete without loop-checking
- Always finds the shortest path



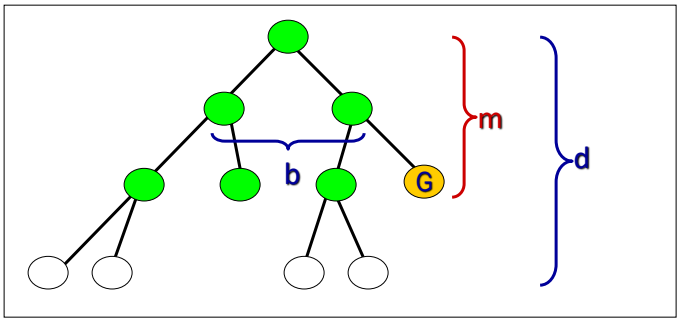
Breedte-eerst kan niet vast komen te zitten in een oneindige tak, dus het zal altijd een oplossing vinden. Ook wanneer we geen loop checking zouden uitvoeren in het algoritme.
Breedte-eerst zal ook altijd de minst diepe oplossing vinden, dus het korte pad tot aan de oplossing.

Conclusie: breedte-eerst is altijd, zonder voorwaarden, compleet.

(Noot: in de volledige cursus gaan we ervan uit dat de branching factor eindig is, bij een spel betekent dit dat vanuit elke spelsituatie een eindig aantal zetten mogelijk is. Dit is de meest realistische situatie, dus de veronderstelling is niet echt een beperking.)

Speed for breadth first

- If a goal node is found on depth **m** of the tree, all nodes up till that depth are created.



- Thus: $O(b^m)$
- note: depth-first would also visit deeper nodes.

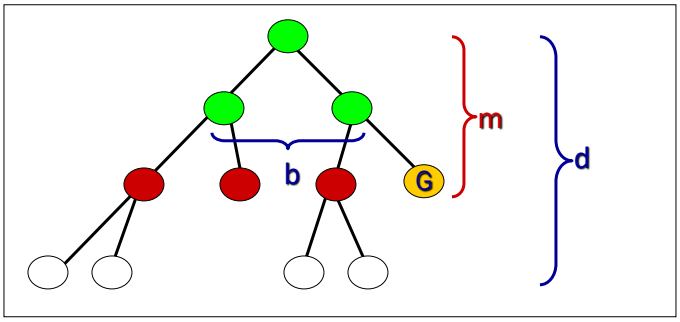
Het maximaal aantal nodes dat gegenereerd moet worden tot een oplossing gevonden is, is het aantal nodes tot op diepte m, dit is de diepte van de minst diepe oplossing.

Bij de bespreking van de rekentijd bij diepte-eerst hebben we al zo'n inschatting gemaakt, stel ze hier zelf op als 'uitstekende oefening voor thuis'.

Conclusie is: $O(b^m)$

Memory for breadth first

- Largest number of nodes in QUEUE is reached on the level **m** of the goal node.



- QUEUE contains all red and yellow (goal) nodes. (Thus: 4).
- In general: b^m
- This usually is MUCH worse than depth-first!

Deze slide heeft weinig extra informatie nodig, de redeneringen werden ook al toegelicht op eerdere slides. Probeer als goede oefening zelf de redenering op te bouwen om tot b^m te komen.

Finally: what algorithm to choose?

	Depth-first	Breadth-first
Completeness	Yes, but…	Yes, always
Speed	$O(b^d)$	$O(b^m)$
Memory	O(b*d)	$O(b^m)$

- Not straightforward to choose between the two algorithms
- Each algorithm has his advantages/disadvantages
- Need for other algorithms!

In de tabel staat een samenvatting van de analyse van de algoritmen die we in dit topic opgesteld hebben.

De vraag is dan welk algoritme we moeten kiezen. Daar bestaat geen eenduidig antwoord op:
- wil je het kortste pad vinden?
- is het computergeheugen groot genoeg om je spelboom te doorzoeken?
- is geheugen de cruciale factor, of rekentijd?
- …

Veel hangt dus af van de aard van de zoekboom. Heb je een zoekboom met oneindige takken, dan gebruik je best breedte-eerst. Qua geheugen schijnt diepte-eerst de beste keuze te zijn. Qua snelheid is breedte-eerst in het algemeen weer de betere keuze...

Er zijn echter algoritmen die het beste van de twee algoritmen, breedte-eerst en diepte-eerst, probeert te combineren. Dit algoritme komt aan bod in het volgende topic.

Questions

?

Coordinates:

Dr. Stefaan Haspeslagh

A.I. Research & Education

Vives University of applied sciences – KU Leuven Association

E-mail: stefaan.haspeslagh@vives.be